



Materia: Seminario de solución de problemas de traductores de lenguaje ll

Profesor: Michel Emmanuel López Franco

Alumno: Alejandro Oñate Navarro

Tarea: ¿Qué es un analizador léxico?

¿Qué es un analizador léxico?

Introducción

Cuando uno programa, escribe texto plano. Para la computadora ese archivo no es más que una tira de caracteres, sin ninguna estructura. El trabajo del compilador es convertir esa tira en algo ejecutable, y no lo hace de un solo golpe: lo hace por fases. La primera de todas es el análisis léxico, y es de la que trata este reporte.

Investigué en qué consiste, para qué sirve dentro del compilador, en qué teoría se apoya y cómo se implementa en la práctica.

Qué es

El analizador léxico —también le dicen *scanner* o *lexer*— es la parte del compilador que lee el archivo fuente carácter por carácter y va agrupando esos caracteres en unidades que sí significan algo. A esas unidades se les llama **tokens**.

En pocas palabras: entra una tira de caracteres sin forma y sale una lista de piezas ya clasificadas. Un identificador, una palabra reservada, un número, un operador, un punto y coma. El scanner no entiende qué hace el programa; nada más lo parte en pedazos que el resto del compilador pueda manejar.

Dónde encaja

El análisis léxico es la primera fase del *front-end* del compilador. El orden completo va así:

1. Análisis léxico — genera los tokens.
2. Análisis sintáctico — acomoda los tokens según una gramática.
3. Análisis semántico — revisa tipos, ámbitos, coherencia.
4. Generación de código intermedio.
5. Optimización.
6. Generación de código objeto.

Algo que me pareció interesante es que en la práctica el léxico no corre solo de principio a fin. Normalmente el analizador sintáctico es el que manda, y cada vez que necesita algo le pide al léxico el siguiente token con una función tipo `getNextToken()` o `yylex()`. Así no hay que guardar todo el programa tokenizado en memoria; se va generando bajo demanda.

Tres conceptos que se confunden

Al leer sobre el tema me topé con tres términos que parecen lo mismo pero no lo son:

Concepto	Qué es	Ejemplo
Token	La categoría. Es lo que se le entrega al sintáctico.	IDENTIFICADOR, NUM_ENTERO, OP_SUMA
Lexema	Los caracteres tal cual venían en el archivo.	contador, 42, +
Patrón	La regla que dice qué lexemas caen en ese token.	[a-zA-Z_][a-zA-Z0-9_]*

Muchos tokens además cargan un **atributo**: el valor del número, el nombre del identificador o su posición en la tabla de símbolos. Sin eso, el token NUM_ENTERO no serviría de nada porque no sabríamos qué número era.

Qué más hace, aparte de tokenizar

Revisando la bibliografía, el scanner termina haciendo varias cosas que no son estrictamente reconocer tokens, pero que le simplifican la vida al resto del compilador:

- Tira a la basura lo que no importa: espacios, tabuladores, saltos de línea y comentarios.
- Registra los identificadores en la tabla de símbolos, o por lo menos crea la entrada que después se llena.
- Lleva la cuenta de líneas y columnas. Sin esto los mensajes de error serían inservibles: "error de sintaxis" a secas no le sirve a nadie.
- Detecta errores léxicos, o sea cadenas que no encajan en ningún patrón del lenguaje.
- En algunos lenguajes, resuelve directivas de preprocesador o expande macros.

La teoría detrás

Aquí es donde se conecta todo lo que vimos en Traductores I. Los tokens de un lenguaje de programación son lenguajes regulares, y por eso se pueden describir con expresiones regulares y reconocer con autómatas finitos.

El camino típico para construir uno es:

1. Escribir cada token como **expresión regular**.
2. Pasar la expresión regular a un **AFN**, con la construcción de Thompson.
3. Convertir el AFN a un **AFD**, con construcción de subconjuntos.

4. **Minimizar** el AFD para bajar el número de estados.
5. Implementar el AFD, ya sea con una tabla de transiciones o directo con condicionales y un switch.

Esto no me resultó ajeno porque en Seminario de Traductores de Lenguajes I diseñé precisamente autómatas finitos para reconocer constantes numéricas enteras en lenguaje C. En ese momento el autómata era el fin del ejercicio; ahora queda claro que era una pieza de algo más grande: ese mismo autómata es el que va a alimentar al analizador sintáctico.

También hay dos reglas para decidir qué hacer cuando varios patrones podrían aceptar la misma entrada:

- **Lexema más largo** (*maximal munch*): gana la coincidencia más larga. Por eso `>=` se lee como un solo operador y no como `>` seguido de `=`.
- **Prioridad de la regla**: si dos patrones aceptan lo mismo con la misma longitud, gana el que se declaró primero. Gracias a esto `while` se reconoce como palabra reservada y no como identificador, aunque cumpla el patrón de identificador.

Errores léxicos

Un error léxico pasa cuando ninguna regla del lenguaje puede reconocer lo que se está leyendo. Los casos más comunes son símbolos que ni siquiera pertenecen al alfabeto del lenguaje, cadenas a las que les falta la comilla de cierre, comentarios que nunca se cierran o números mal formados como 12.3.4.

Lo normal es no abortar en el primer error. Se usa el llamado **modo pánico**: se descarta el carácter que causó el problema y se sigue desde el siguiente, para juntar la mayor cantidad de errores posible en una sola pasada. Es más útil para quien programa recibir cinco errores de golpe que uno a la vez.

Un ejemplo

Tomando esta línea de C:

```
c  
suma = valor1 + 25;
```

El analizador léxico devolvería:

Lexema	Token	Atributo
suma	IDENTIFICADOR	apuntador a tabla de símbolos
=	OP_ASIGNACION	—

Lexema	Token	Atributo
valor1	IDENTIFICADOR	apuntador a tabla de símbolos
+	OP_SUMA	—
25	CONST_ENTERA	valor 25
;	PUNTO_Y_COMA	—

Lo que hay que notar es que los espacios desaparecieron por completo y que ya nada es "texto": son seis categorías. De aquí en adelante el compilador ya no trabaja con caracteres.

Herramientas

No siempre hay que escribir el autómata a mano. Existen generadores a los que se les da un archivo con las expresiones regulares y las acciones, y ellos construyen el AFD y escupen el código:

- **Lex / Flex** — genera C. Es el clásico en Unix y el que más se usa en cursos de compiladores.
- **JFlex** — el equivalente en Java, normalmente combinado con CUP.
- **ANTLR** — genera léxico y sintáctico para varios lenguajes destino desde una sola gramática.
- **PLY** — la versión de Lex y Yacc para Python.

Vale la pena entender primero cómo se construye a mano, porque cuando la herramienta falla hay que saber qué está haciendo por dentro.

¿Por qué no juntarlo con el análisis sintáctico?

Me pregunté esto mientras investigaba, porque en teoría se podrían fusionar las dos fases. Las razones para mantenerlas separadas son prácticas:

- La gramática del sintáctico se vuelve mucho más limpia si no tiene que describir espacios ni comentarios.
- La lectura de caracteres y el manejo de buffers se pueden optimizar aparte, que es donde de verdad importa el rendimiento.
- Los detalles de codificación de caracteres quedan encerrados en un solo módulo.
- Si mañana hay que agregar un operador nuevo, solo se toca el scanner.

Conclusiones

El analizador léxico es la entrada del compilador y, aunque su tarea suena sencilla, es la que le pone la primera capa de estructura a un archivo que en crudo no tiene ninguna.

Lo que me queda más claro después de investigarlo es que esta fase es la aplicación más directa de la teoría de lenguajes formales. Las expresiones regulares y los autómatas finitos dejan de ser un ejercicio de pizarrón y se vuelven una herramienta concreta. También queda claro que todo lo que venga después depende de que esta parte esté bien hecha: si el scanner clasifica mal un token o pierde el conteo de líneas, el error se arrastra hasta el final y se manifiesta en lugares que no tienen nada que ver con la causa real.

Referencias

- Aho, A. V., Lam, M. S., Sethi, R. y Ullman, J. D. (2008). *Compiladores: principios, técnicas y herramientas* (2ª ed.). México: Pearson Educación.
- Loudon, K. C. (2004). *Construcción de compiladores: principios y práctica*. México: Thomson.
- Hopcroft, J. E., Motwani, R. y Ullman, J. D. (2007). *Introducción a la teoría de autómatas, lenguajes y computación* (3ª ed.). Madrid: Pearson Addison-Wesley.
- Levine, J. (2009). *flex & bison: Text Processing Tools*. Sebastopol, CA: O'Reilly Media.